Java Class Primer

Class anatomy: fields, constructor, methods, extends, implements, annotations

Interface + Base Class

```
// An interface declares WHAT a class must do, not HOW.
// Implementors must provide all non-default methods.
public interface Soundable {
  void makeSound();
    // no body — implementor decides
  default void breathe() {
    // default = optional to override
    System.out.println("breathing");
  }
}
// A class bundles data (fields) and behavior (methods).
public class Animal {

  private String name;
    // private: only accessible inside Animal
  private int    age;
  public static int count = 0;
    // static: one copy shared by ALL instances
  public final  String species;
    // final: assigned once, never changed
  // Constructor: runs once on new Animal(...)
  public Animal(String name, int age, String species) {
    this.name    = name;
      // this.name = field, name = parameter
    this.age     = age;
    this.species = species;
    count++;
      // increment shared counter
  }
  public String getName() {
    return name;  // getter: reads a private field
  }
  public void birthday() {
    age++;  // void: returns nothing
  }
  public String describe() {
    // can be overridden by subclasses
    return name + " is a " + species;
  }
}
```

Subclass + Usage

```
// extends: Dog IS-A Animal
// implements: Dog fulfills the Soundable contract
public class Dog extends Animal implements Soundable {

  private String breed;
    // Dog-specific field
  public Dog(String name, int age, String breed) {
    super(name, age, "Dog");
      // calls Animal constructor first
    this.breed = breed;
  }
  // @Override: compiler checks method exists in Animal
  @Override
  public String describe() {
    return super.describe() + ", breed: " + breed;
  }
  // Required by Soundable — must be provided
  @Override
  public void makeSound() {
    System.out.println("Woof!");
  }
  // breathe() not overridden — uses Soundable default
}

// Using the classes
Animal a = new Animal("Felix", 3, "Cat");
Dog    d = new Dog("Rex", 5, "Labrador");
Animal r = new Dog("Buddy", 2, "Poodle");
  // Dog IS-A Animal — valid assignment

System.out.println(a.describe());
  // Felix is a Cat
System.out.println(d.describe());
  // Rex is a Dog, breed: Labrador
d.makeSound();
  // Woof!
d.breathe();
  // breathing  (from default)
System.out.println(Animal.count);
  // 3  (static field, shared by all)
```

Concept Callouts

Class Declaration

public class Name { }
public = accessible from other files
Filename must match class name exactly

Fields

private = only this class can read/write
static = one copy shared by all instances
final   = assigned once, never reassigned
static final = constant (team standard: prefix k)

Constructor

Runs once when you write new ClassName(...)
this.field = field disambiguates from parameter
super(...) calls the parent class constructor

Methods

void = returns nothing
String/int/etc. = must return that type
public/private controls who can call it

extends (one parent only)

Dog IS-A Animal: inherits all fields and methods
Use @Override to replace a parent method
super.method() calls the parent version

implements (multiple allowed)

Promises to provide all interface methods
default methods are optional to override
A class can implement multiple interfaces

Annotations

Metadata read by the compiler or build tools
@Override verifies method exists in parent
Other annotations can trigger code generation

Class Shell | Fields | Constructor | Methods | AKit Infrastructure | IO / Delegation | Comments | Visualization

IO Interface + Hardware Implementation

```java
// ArmConstants.java — static constants only, no logic
public final class ArmConstants {
  public static final double kMinAngleDeg    = 0.0;
  public static final double kMaxAngleDeg    = 180.0;
  public static final double kStowedAngleDeg = 180.0;
  public static final double kLowAngleDeg    = 135.0;
  public static final double kHighAngleDeg   = 90.0;
}

// ArmIO.java — interface: what any ArmIO must do
public interface ArmIO {
  @AutoLog
    // generates ArmIOInputsAutoLogged at compile time
  public static class ArmIOInputs {
    public double commandedAngleDeg = kStowedAngleDeg;
  }

  public default void updateInputs(ArmIOInputs inputs) {}
    // empty body = safe no-op for replay
  public default void setAngle(double angleDeg) {}
}

// ArmIOXRP.java — hardware implementation
public class ArmIOXRP implements ArmIO {
  private final XRPServo armServo =
      new XRPServo(kArmServoDeviceNumber);

  @Override
  public void updateInputs(ArmIOInputs inputs) {
    // no sensor feedback on XRP servo
    // commandedAngleDeg tracked by subsystem
  }

  @Override
  public void setAngle(double angleDeg) {
    armServo.setAngle(angleDeg);
      // direct hardware call — no logic here
  }
}
```

Simulation Implementation + Wiring

```java
// ArmIOSim.java — simulation implementation
// Symmetric with ArmIOXRP: no physics for servo
public class ArmIOSim implements ArmIO {
  @Override
  public void updateInputs(ArmIOInputs inputs) {
    // no sensor feedback; commandedAngleDeg tracked by subsystem
  }
  @Override
  public void setAngle(double angleDeg) {
    // no hardware to command in sim
  }
}
```

RobotContainer — IO Selection

```java
// RobotContainer.java — IO selection only
public class RobotContainer {
  private final Drive drive;
  private final Arm arm;
  private final LoggedDashboardChooser<Command> autoChooser
      = new LoggedDashboardChooser<>("Auto Choices"); // replaces SendableChooser
  public RobotContainer() {
    switch (Constants.currentMode) {
      case REAL:
        // physical XRP hardware
        drive = new Drive(new DriveIOXRP());
        arm   = new Arm(new ArmIOXRP());   break;
      case SIM:
        // WPILib simulator
        drive = new Drive(new DriveIOSim());
        arm   = new Arm(new ArmIOSim());   break;
      default:
        // log replay — anonymous empty impl
        drive = new Drive(new DriveIO() {});
        arm   = new Arm(new ArmIO() {});   break;
    }
    configureButtonBindings(); configureAutonomous();
  }
}
```

Class Shell | Fields | Constructor | Methods | AKit Infrastructure | IO / Delegation | Comments | Visualization

Java Class Structure & AdvantageKit Reference

The Subsystem: Arm.java with all class categories annotated

Arm.java — Fields, Constructor

```java
// Arm.java — the subsystem
public class Arm extends SubsystemBase {

  // ── AKit infrastructure (required) ──
  private final ArmIO io;
  private final ArmIOInputsAutoLogged inputs
      = new ArmIOInputsAutoLogged();

  // ── Control math (motor-driven subsystems) ──
  // private final ProfiledPIDController controller
  //   = new ProfiledPIDController(kP, 0, kD,
  //       new TrapezoidProfile.Constraints(
  //           kMaxVelDegPerSec, kMaxAccelDegPerSec));
  // private final ArmFeedforward feedforward
  //   = new ArmFeedforward(kS, kG, kV);

  // ── State tracking ──
  // @AutoLogOutput
  // private double setpointDeg = 0.0;
      // logs automatically every loop

  // ── WPILib utilities ──
  // private final Alert encoderAlert = new Alert(
  //     "Arm encoder disconnected.", AlertType.kError);

  // ── Visualization ──
  private final LoggedMechanism2d mechanism
      = new LoggedMechanism2d(3, 3);
  private final LoggedMechanismRoot2d root
      = mechanism.getRoot("ArmPivot", 1.2, 0.18);
  private final LoggedMechanismLigament2d armLigament
      = root.append(new LoggedMechanismLigament2d(
          "Arm", feetToMeters(3), 0));

  public Arm(ArmIO io) { this.io = io; }
    // IO impl chosen by RobotContainer
```

Arm.java — Methods

```java
  @Override
  public void periodic() {
    io.updateInputs(inputs);
      // always first
    Logger.processInputs("Arm", inputs);
      // always second

    armLigament.setAngle(180 - inputs.commandedAngleDeg);
    Logger.recordOutput("Arm/Mechanism2d", mechanism);
    Logger.recordOutput("FinalComponentPoses",
        new Pose3d[] { ... });

    // Motor-driven subsystems run PID+FF here:
    // double pid = controller.calculate(
    //     inputs.positionDeg, setpointDeg);
    // double ff = feedforward.calculate(
    //     controller.getSetpoint().position,
    //     controller.getSetpoint().velocity);
    // io.setVoltage(pid + ff);
  }

  // Clamp at subsystem boundary
  public void setAngle(double angleDeg) {
    double clamped = MathUtil.clamp(angleDeg,
        ArmConstants.kMinAngleDeg,
        ArmConstants.kMaxAngleDeg);
    io.setAngle(clamped);
      // delegate to IO — never call hardware directly
    inputs.commandedAngleDeg = clamped;
      // sync logged value (no sensor on servo)
  }

  public void stop() {
    setAngle(ArmConstants.kStowedAngleDeg);
  }
  public double getCommandedAngleDeg() {
    return inputs.commandedAngleDeg;
  }
}
```

Callouts

AKit Infrastructure

ArmIO io — interface reference injected via constructor.
ArmIOInputsAutoLogged — generated class from @AutoLog.
Never use ArmIOInputs directly in the subsystem.

Control Math (commented out)

ProfiledPIDController + feedforward belong here for any motor-driven subsystem.
Drive will implement this in a later lesson.

Visualization

LoggedMechanism2d renders arm in AdvantageScope.
Logger.recordOutput() for computed/derived values.
FinalComponentPoses drives the 3D robot model.

Constructor Rule

Subsystem does not know if it is REAL, SIM, or REPLAY.
RobotContainer injects the correct IO implementation.

periodic() — always in this order

1. io.updateInputs(inputs)
2. Logger.processInputs("Arm", inputs)
3. All logic reads from inputs.*, never hardware

IO Delegation Rule

io.setAngle() executes the command.
The IO layer never decides — it only executes.
Clamping belongs at the subsystem boundary.

inputs.commandedAngleDeg

Written by subsystem, not by updateInputs, because the XRP servo has no position sensor.
processInputs() logs this value every loop.

Class Shell | Fields | Constructor | Methods | AKit Infrastructure | IO / Delegation | Comments | Visualization

Java Class Structure & AdvantageKit Reference

Where Logic Lives + Key Types Cheat Sheet + Common Mistakes

Where Logic Lives

What	File
Hardware objects (motors, servos, sensors)	ArmIOXRP.java only
Unit conversion (volts to motor %)	ArmIOXRP.java only
Sensor reading / updateInputs()	ArmIOXRP.java (impl); ArmIO.java (sig)
Reading sensor state in subsystem logic	inputs.* in Arm.java
PIDController / ProfiledPIDController	Arm.java
Feedforward (ArmFeedforward, etc.)	Arm.java
@AutoLogOutput state fields	Arm.java
Timer, Alert	Arm.java
Clamping / safety logic	Arm.java (subsystem boundary)
Coordinating two subsystems	Command class only
Choosing REAL vs SIM vs REPLAY	RobotContainer constructor
Port numbers, setpoints, PID gains	ArmConstants.java
Logger.start()	Robot.robotInit() — line 1
CommandScheduler.getInstance().run()	Robot.robotPeriodic()

Key Types Cheat Sheet

Type	Package	What it does
LoggedRobot	o.l.junction	Replaces TimedRobot as Robot.java base class
Logger	o.l.junction	start(), processInputs(), recordOutput()
SubsystemBase	e.w.f.wpilibj2.command	WPILib base; registers periodic() each loop
@AutoLog	o.l.j.autolog	On inputs class; generates *AutoLogged
@AutoLogOutput	o.l.junction	On subsystem field; auto-logs as output every loop
@Override	Java built-in	Verifies method exists in parent/interface
PIDController	e.w.f.math.controller	Feedback control for velocity or position
ProfiledPIDController	e.w.f.math.controller	PID + trapezoidal motion profile
SimpleMotorFeedforward	e.w.f.math.controller	Feedforward for flywheels and drive wheels
ArmFeedforward	e.w.f.math.controller	Feedforward for rotating arms (gravity-aware)
ElevatorFeedforward	e.w.f.math.controller	Feedforward for vertical elevators
LoggedDashboardChooser	o.l.j.networktables	Logged replacement for SendableChooser
Timer	e.w.f.wpilibj	Timing events inside a subsystem
Alert	e.w.f.wpilibj	Persistent fault notification in DS + AScope

Common Mistakes

Mistake	Fix
Logger.getInstance().start()	Logger.start() — static method, no getInstance
ArmIOInputs in Arm.java	Use ArmIOInputsAutoLogged (compiler-generated class)
servo.getAngle() in Arm.java	Read inputs.commandedAngleDeg instead
Hardware objects in Arm.java	Hardware only in ArmIOXRP.java
PIDController in ArmIOXRP.java	Control math belongs in Arm.java
Clamping inside IO setAngle()	Clamp at subsystem boundary; IO just executes
updateInputs() after logic	Must be the first two lines of periodic()
SendableChooser for auto	Use LoggedDashboardChooser — selection is an input
Subsystem coord in periodic()	Move coordination logic into a Command class